Room Config Builder

Operation guide for the Deployment Configurator

Building rooms, drawing them, pricing them - and teaching the app your own rules.

What is in here

Start here

What this program is for

You have a room. It has a projector, a couple of cameras, a DSP, a switcher and a touch panel, and somewhere on the processor there is a `config.json` that ties all of it together. Editing that file by hand works right up until the moment it doesn't: a key gets misspelled, a device block is left behind after the room shrank, and the room comes up wrong on a Monday morning.

The Room Config Builder is the tool that stands between you and that file. You work through tabs that show each setting with a proper label, a description and the right kind of control, and the app writes the JSON. When the room is finished you can push it straight to the processor over SFTP.

But it does more than edit the config now. The same room description drives:

- a **control schematic** - what talks to the processor, and how
- an **AV signal flow** - what plugs into what, drawn from the switcher numbers you already typed
- **rack elevations**, **floor plans** and a **cable schedule**
- a **cost estimate**, built from a device catalog you keep yourself

And a folder of rooms is a **building**: one master parts list, a quote request per vendor, the purchase orders they turn into, a delivery log, a calendar that says when each part has to be bought, a matrix of who furnishes and who installs each line of scope, and a refresh plan that says which year the room comes round again. A folder of buildings is an **estate**.

Most of that draws itself. You describe the room once, on the tabs you were going to fill in anyway, and the drawings and the numbers follow - and nothing on any of them is typed twice. A model changed on the drawing changes the quote, the rack, the cable schedule and the twenty-year plan, because all four are readings of one description rather than four documents to keep in step.

The idea worth knowing

The app tries very hard not to have opinions baked into it.

Almost everything it knows about *your* rooms lives in plain JSON files that sit next to the program, and every one of them now has an editor inside the app:

File	What it decides	Where you edit it
`ui_schema.json`	How each config key looks on screen: label, description, dropdown or switch, which device families exist at all	**Schema** tab

File	What it decides	Where you edit it
`av_flow_rules.json`	How a room turns into a drawing: which box a config key means, what goes between two ends that don't match, what hangs off the USB switcher	**Flow Rules** tab
`av_devices.json`	The equipment catalog: connectors, rack units, power draw, price	**Catalog** tab
`key_map.json`	How old files are translated to current key names on load	Text editor (reference in this guide)
`labor_rates.json`, `base_costs.json`	What an hour costs, and the rate card the estimate and the refresh plan fall back to	Dialogs on the **Cost** tab, and the campus report's **Current models** tab

If the app does something you don't want - puts the wrong receiver in a run, calls a field by a name your team doesn't use, misses a key you just added to the template - the fix is usually a rule or a schema entry, not a phone call and a wait for the next build.

> Nothing in this guide requires you to edit JSON by hand. Everything described here can be done from a tab in the app. The reference chapters at the back are for when you would rather see the file.

The tabs

They run down the left-hand rail in roughly the order you use them.

Tab	What it's for
Project	A whole building: rooms quoted together, one master parts list, the vendors, the orders, the deliveries, the calendar and the refresh plan.
Cost	The estimate for the room you're on: equipment, cable, labor, tax and fees.
Lifecycle	What this room already has, how old it is, and the year each position falls due.
Wizard	The room's identity - building, room number, name - and how many of each device it has.
Devices	One sub-tab per device: model, IP, connection type, module, and whatever else the schema says that family has.
System	Everything else in `SYSTEM_SETUP`: switcher inputs and outputs, panel behavior, outlet names.

Tab	What it's for
Raw JSON	The whole config as text, for when you want to see or paste it. Apply writes it straight back to the working file.
Schematic	The control topology: what the processor talks to, over what, drawn automatically.
AV Flow	The signal flow: what plugs into what, also drawn automatically.
Floor Plan	Where things physically are, with callouts on a sheet.
Cabling	Low-voltage runs that aren't signal - screen triggers, shades, and a run schedule.
Racks	Rack elevations, front and rear, with clearances and a parts list.
Catalog	The device catalog every drawing and every price is built from.
Schema	What the Devices and System tabs look like - labels, types, descriptions, device families.
Flow Rules	How the AV Flow decides what to draw.
App Config	Where the files live, the theme, the pricing tier, the SFTP settings and the active deployment target.

Project and Cost are at the top because that's the order the work goes in: open the job, see what the building costs, then go into the rooms that make it up. **Lifecycle** sits with them because it is the same question at the other end of the room's life - what it costs to put in, and what year it has to be put in again.

Two things live above the rail rather than on it. **Help** is on the question mark beside the gear, on every screen; **Campus** appears on the job banner once a project is open, and puts several buildings on one sheet. Campus over project over room, three levels, each with its own way back out.

The **Catalog**, **Schema** and **Flow Rules** tabs work with no room open at all - they're about the app, not about one room. So does **Project**, for a different reason: it points at room *files* and prices them off disk, so reviewing a building's quote never means opening a room.

The rail always fits the window. Shrink the window and the tabs get tighter; get it small enough and the words come off and you're left with icons you can hover for the name. You should never have to scroll it to find a tab.

Setting up, once

Point it at a folder

The quickest setup is one folder holding everything the app reads. On first launch you'll be asked where that is; you can change it any time from **App Config**.

Every other path defaults to that Root Folder, so a folder that looks like this needs no other configuration:

```
Z:\AV\RoomConfigBuilder\
    config.json            the template a new room starts from
    processors.json        the rooms you can deploy to (name + IP)
    buildings.json         building names and their codes
    ui_schema.json         how config keys appear on screen
    key_map.json           translation of legacy key names
    av_devices.json        the equipment catalog and price list
    av_flow_rules.json     how a room draws itself
    labor_rates.json       what an hour costs
    base_costs.json        fallback prices by category
    devices\               the Python control modules
    documentation\         PDF manuals, shown in the app
```

Any file that isn't there simply isn't used: with no `av_flow_rules.json` the app draws with its built-in rules, with no `key_map.json` nothing is translated, and so on. Nothing breaks because a file is missing.

> **Put the shared ones on a share.** `av_devices.json`, `ui_schema.json` and `av_flow_rules.json` describe how your shop works, not how your laptop is set up. One copy on a network drive means two engineers drawing the same room draw it the same way. The catalog even merges another engineer's edits rather than overwriting them.

Your settings

App Config also holds the things that are yours rather than the department's: the theme (Classic or the Auris HUD look), text size, SFTP username and port, and the **Active Deployment Target** - the room the next upload or download talks to.

It also holds the **pricing tier**, which is not quite a preference: every price in this app is published at two of them - MSRP (list) and the education / institutional price - and the one picked here is the one every estimate, every replacement figure and every report is quoted at. A catalog entry or a base cost card with only one of the two filled in falls back to the other rather than reading as free, and says that it fell back.

That target now clears itself whenever you open or create a different config. It's the quietest mistake this app could make - BSS 103's config uploaded to SSC 210's processor, with the tab that says so two screens away - so you're asked to pick the room again rather than inheriting the last one. Downloading from a processor keeps the target, because that config *came* from there.

A room, start to finish

This is the whole workflow in one page. Every step has a chapter of its own later.

1. **Start the room.** *New Config* builds from your template with every device count at zero. *Open Existing Config* loads a file from disk. *Download from Processor* pulls `/config.json` over SFTP and asks where to keep the working copy. The New Room dialog asks **Start from the cost estimator** first, because that's how a room actually starts: pick the devices out of the catalog with quantities and a running total, and they become the boxes on the signal flow, the gear in the racks and the lines on the estimate.
2. **Say what the room is.** On the **Wizard**, pick the building and type the room number, then set the device counts. Blocks and tabs appear and disappear as you change them.
3. **Fill in the devices.** On **Devices**, each one gets its model, IP or COM port, and control module. *Check Defaults* tells you what a block is missing; *Check Module Defaults* tells you where it disagrees with the driver.
4. **Fill in the system.** On **System**, the switcher input and output numbers, the panel options, the outlet names. This is the part the drawings read.
5. **Look at the drawings. Schematic** and **AV Flow** have already drawn themselves from what you typed. Fix anything that looks wrong - by dragging, or by correcting the config, or by editing a rule.
6. **Rack it and place it. Racks** for the elevations, **Floor Plan** for where things are in the room.
7. **Price it. Cost** builds the estimate from the drawing and the catalog.
8. **Hand it over.** *Save All* writes every drawing, report and workbook into one folder. *Upload to Processor* pushes the config itself.

What gets written where

The config is one file. Everything else the app knows about the room lives in small companion files beside it, so the cost estimate and the rack elevation aren't fighting over the same document:

```
BSS103_config.json                    the config itself
BSS103_config_av_flow.json            the signal flow drawing
BSS103_config_racks.json              rack elevations
BSS103_config_floor_plans.json        floor plan sheets and locations
BSS103_config_cabling.json            low-voltage runs
BSS103_config_cost.json               the estimate
BSS103_config_control_schematic.json  the control schematic layout
BSS103_old_config.json                the pristine original, from the first load
```

Opening, converting and saving

Getting back to a file you had open

The app remembers the last ten rooms, the last ten jobs and the last ten campuses it opened or saved. They are on the **Open Recent** menu beside Open in the title bar, and on the start screen under the Open button.

Three lists, not one: a room, a job and a campus all live as `.json` files in the same folders, and one mixed list of thirty is a list you have to read rather than glance at. Each kind keeps its own ten, headed and most recent first.

Every open counts, not just the ones that went through the file dialog - a room reached from the Project tab lands on the list the same way as one picked by hand. An entry is a pointer, never a copy, so choosing one re-reads the file off disk exactly as Open would.

Saving counts too, because half the documents here are never opened at all: a room comes out of the wizard, a job out of New Project, an estate out of a list you assembled by hand. Each of those goes on the list the first time it is written, and a *Save As* moves the entry to the file you are now working from. The one exception is *Save All to a room folder* - that walks every room of the job on your behalf, and nine rooms swept through in one press would push a week of real work off the list, so the walk is not recorded.

A file that has been moved, renamed or archived stays on the list, drawn struck through; choosing it offers to drop the line rather than pretending it opened. *Clear the list* forgets all of them and touches no file. The lists are kept in `app_config.json` with the rest of your settings, so they survive an update.

What happens when a file loads

Old configs don't look like new ones. Rather than making you fix them by hand, the app runs every load through the same pipeline, in this order:

1. **Key mapping** - `key_map.json` renames legacy sections and properties (`CAMERA1DEVICE.IPADDRESS` becomes `CAMERADEVICE_1.ip_address`). It runs first so everything after it sees current names.
2. **Automatic backup** - the original text, untouched, is written next to the working file as `<ROOM>_old_config.json`. The name comes from the room identity, which is why mapping runs first.
3. **Template migration** - baseline `SYSTEM_SETUP` values the file is missing are added. Existing values are never overwritten.
4. **Building code normalization** - a `gve_bldg` holding a full building name is converted to its code.
5. **Device defaults fill** - device blocks missing their family's baseline properties get them (switch this off in App Config if you'd rather it didn't).
6. **Auto room name** - `gui_full_room_name` is rebuilt in Title Case.

7. **Audits** - more device blocks than the count allows, or keys that aren't in the template, are reported. Nothing is changed.

Everything the pipeline did shows up in the dialog afterwards, in the migration log, and in a change log written next to the backup. If you don't like what it did, the pristine original is right there.

Getting the config back out

- **Export Config Locally** - a save dialog, named from the building and room (`BSS_103_config.json`).
- **Upload to Processor (SFTP)** - pushes to `/config.json`, optionally with a companion file such as `Whereused.csv`. The dialog has the same searchable room picker as the download.
- **Apply Changes** on the Raw JSON tab - parses the text back into memory *and* writes the working file, so Apply doubles as Save.

Every path out writes keys sorted in natural order, so `PROJECTORDEVICE_2` comes before `PROJECTORDEVICE_10` and two exports of the same room diff cleanly.

Devices

Each device gets a sub-tab: its model, how the processor reaches it, and the python module that drives it. *Check Defaults* tells you what a block is missing; *Check Module Defaults* tells you where it disagrees with the driver.

When the model and the driver don't match

A device block names a product and names the driver that talks to it, and nothing used to keep the two together. Retype the model, or swap the box from the **Cost** tab, and the module underneath went on naming a driver for the product that used to be there - with every field filled in, so the block read as finished. That's the config nobody re-checks, and it commissions the room as a device the room doesn't contain.

A red banner now sits at the top of the device, above the two fields that fix it, in two cases:

- **no python module is set** for a model that has one - nothing can talk to the device, so the room won't commission
- **the module is set, says which models it covers, and this isn't one of them**
  - the banner names what that driver actually drives

Pick a module (or correct the model) and it clears. It's read from the config rather than remembered by the page, so it survives leaving the tab, saving, and reopening the room - and it's what a swap made on the Cost tab leaves behind.

It stays quiet where it can't know: a device with no model yet, a driver that never declared which models it covers, and a model the driver does list under a different capitalization.

A driver that lists no models

"A driver that never declared which models it covers" is worth its own section, because it's the quietest fault in the app. At the top of each python driver is a short block - `DEVICE_INFO` - naming the models it drives, the kind of equipment they are, and how the box is reached. That block is where the Model dropdown comes from, what picking a model fills a device in with, and what *Check Module Defaults* measures a room against.

A driver without one still loads and still answers. Its products simply never appear anywhere, which looks exactly like the app not supporting them.

App Config -> What each driver declares... (also on the Devices tab, beside the module box) lists every driver the app has read and marks the ones with no block.

- **Read the file** fills the form in from what the driver actually says about itself: the models out of its own `self.Models` dict, and the baud rate, protocol and connection styles out of the wrapper classes at the bottom of it. Anything already typed in is left alone, so it's safe to press on a half-finished block.

- **The one thing a driver can never say is its network port.** Every driver is told which port to use rather than declaring one, so the port is left blank and has to come off the manufacturer's communication sheet. The rest of the blanks are site facts - an address, a COM port, a password - and are left for the room.
- Panel object names, the keep-alive and the credentials are seeded from what every other driver of that family in the folder already uses, and the keep-alive is checked against the commands this driver actually has.
- **Save to the .py** shows the exact python first. Only that one block in the file is touched, and the whole folder is re-read afterwards, so the Model dropdown is looking at the file as it is now.

The drawings

Control Schematic

This one answers a single question: *what does the processor talk to, and how?*

It draws itself from the config every time you open it. Network devices go to a Network IDF box and up to the processor; serial devices get a direct line labeled with their COM port; relay-controlled screens get a relay line; the touch panel is drawn as a window with one tab per GUI page. Each kind of connection has its own color, and the legend under the drawing explains them.

Things worth knowing:

- **192.x drops** aren't on the building network - they're on the AV LAN. The toolbar has a dropdown for where those actually land in your buildings: the IDF, the processor's own second NIC, or a switch you draw yourself. The touch panel has its own dropdown, because a panel on PoE off the building switch is as ordinary as one beside the processor.
- **Add device** (in Edit mode) puts a box on the drawing for equipment the control system doesn't talk to at all - the building switch, a UPS, the room PC. They're drawn as related equipment, dashed rather than solid, so nobody reads them as controlled devices.
- **Save Layout** keeps your dragging next to the config; **Reset Layout** puts every box back where the auto-layout wants it.

Recreate from config

Sometimes the drawing has drifted far enough that tidying it costs more than starting again - a room got re-scoped, half the devices changed, and there are hand-drawn lines to boxes that no longer exist.

Recreate from config throws the layout away and lets the drawing rebuild itself: every box back at its automatic spot, every generated line back including any you had hidden, and hand-added boxes removed. It asks first and tells you exactly what it will take with it, and one press of Undo brings it all back.

Two things it deliberately keeps: your line **colors** (they have their own *Reset all* on the Colors dialog) and the **192.x / touch panel landing** choices, because where the room's drops actually go is a fact somebody recorded, not drift. The one exception is a landing pointing at a hand-added box that's being removed - that goes back to the IDF, since the drops would otherwise land on nothing.

AV Flow

The AV Flow is the signal flow: sources, the switcher, displays, and every lead between them.

It draws itself too, from two things you have already typed. The **device blocks** say what the room has. The **input and output numbers** in `SYSTEM_SETUP` say what plugs in where: `input_pc: "1"` means the room PC is on switcher input 1, `output_proj_1: "3B"` means display 1 is on output 3's B connector. The app places the

boxes, finds the right connector on each end, and draws the lead.

It also puts in the boxes nobody writes down because everybody knows they're there:

- A **DTP receiver** behind a display whose matrix output is twisted pair and whose socket is HDMI. That run is two cables and a box, and a drawing that joins them directly is drawing a cable that can't be bought - and an estimate taken off it is missing a $600 box per display.
- A **DTP transmitter** beside a camera on a DTP input, which is the same fact read the other way round.
- The **expansion bus** between a switcher with a DSP in it and the DMP racked beside it, rather than an analog lead into a socket nobody patches.
- The **USB chain** into a USB switcher - see below.
- **Power leads** from the controller, worked out from the outlet names you typed on the System tab. Outlet 1 says "PC", so it goes to the PC.

Every one of those is a rule you can change on the **Flow Rules** tab. Nothing in this list is compiled in.

A room with no switcher

A huddle space has no matrix: one panel with a couple of things plugged into the back of it, which is what `dev_source_control: Display` says. There the `input_` keys are not matrix ties - they're sockets on the display - so the display stands in for the switcher and every source is drawn onto it.

Write the value the way it's printed on the panel. `input_pc: "HDMI 1"`, `input_wireless: "HDMI 2"`, or just `"1"` and `"2"` - all of them resolve, and so does `"HDBaseT"`. The kind and the number both have to match: `HDMI 2` never lands on `HDMI 1`, and never on the DisplayPort socket beside it. A socket the panel doesn't have is reported rather than guessed at, and a socket that already has a lead on it - the meeting bar, usually - is left alone and the disagreement reported.

The `output_` keys mean nothing in such a room: the run stops at the panel, so there's no output side to number. They're ignored rather than resolved against a box that can't answer them.

The USB switcher

Nothing in the config says what plugs into a USB switcher - `dev_usb_switchers` is a count, and there's no input number for a USB lead anywhere in the file. So the drawing used to show a Toggle with nothing plugged into it, and the conferencing path stopped at the DSP.

It's now drawn from the way these rooms are actually wired:

Connector	What lands on it
USB DEVICE 1	the DSP's USB output - the microphone mix
USB DEVICE 2	the AV Bridge's USB output - the room's picture
USB DEVICE 3	the document camera's USB output

Connector	What lands on it
USB HOST 1	the room PC

Those are fixed positions, not a queue. A room with no doc cam leaves DEVICE 3 empty rather than sliding the AV Bridge onto it, because the port a lead lands on is what the tech reads off the drawing. Anything already plugged in by hand is left exactly as you left it. And if your rooms are wired differently, that table is four lines on the Flow Rules tab.

Working on the drawing

- **Edit** mode lets you drag boxes, draw cables between ports, add bends to a run, and open any box to correct its connectors.
- **Place all from config** adds every config device that isn't on the canvas - including any you deleted earlier.
- **Draw the routing from config** shows you what the config's numbers would draw, tie by tie, with a reason for anything that didn't resolve, before it draws it.
- **Recreate from config** starts over: every box and every cable removed, then the room drawn again from the config and the current rules. It asks first, it is one press of Undo, and it keeps the rack rails of devices that come back - re-reading the config is no reason to unrack the room.
- **Room type presets** stamp a whole standard room onto the canvas, wiring and all, for the builds you do over and over.

Reading the routing report

Some ties can't be drawn, and the app would rather tell you than guess. Press *Draw the routing from config* and you get a line per tie: what the config said, what it resolved to, and - for the ones that didn't - why. Typical reasons:

- *"No input on the switcher is labeled 7"* - the number doesn't match any connector on that model. Usually the catalog entry needs its connectors correcting.
- *"output_proj_2 - this room has 1 display, so there is no PROJECTORDEVICE_2"*
  - dead config left over from when the room was bigger.
- *"Every input on the recorder that could take 2 is already fed"* - the box has one socket and the config asks for two feeds.

Racks, plans, cabling and money

Racks

Drag anything on the drawing into a frame and it takes the rails its catalog entry says it needs. The elevation shows front and rear, shades the rails a model wants left clear (the amp that vents upward, the drawer whose lid opens), and warns rather than refuses - the person in front of the frame knows things the catalog doesn't.

Rack hardware that isn't a device - vent plates, shelves, drawers, blanks - is added from the parts list and counts on the estimate like anything else.

Floor Plan

A sheet per room view, with callouts pointing at named places: Instructor station, Front wall, Ceiling, Equipment rack. Every device on the AV Flow can be given a location, and the callouts, the reports and the cable schedule all use the same names.

Cabling

The low-voltage runs that aren't signal flow: screen triggers, shade control, the runs between places in the room. It produces a run schedule with lengths, so somebody can pull cable off it.

Cost

The estimate reads the drawing, not a list you maintain by hand. Every box on the AV Flow is priced from the catalog; every cable is priced by length; labor comes from the rate card, tax and fees from the settings on the tab.

Two things it will tell you rather than quietly getting wrong:

- devices on the drawing whose model the catalog doesn't price, counted so you know how much of the estimate is guesswork
- devices with no control module, so a room isn't quoted as finished when part of it can't be driven

Spares

Equipment rows read **Qty · Spares · Total**, the same three the cabling table has and for the same reason. The drawing says how many the room has; a job often buys one more than that, and the spare is real money no drawing will ever account for.

Type it in the **Spares** box on the line itself rather than adding a second line. It's the same product at the same price, and splitting it out is how a quote ends up with two prices for one box - and how the spare gets left behind when somebody swaps the model. The reports print the split ("3 drawn + 1 spare") so the count never reads as a mistake.

A line you added on this page already has an editable quantity of its own, so it has no spares box - two boxes meaning the same thing on one row is how a number gets typed into the wrong one.

Editing a part without leaving the page

A price rise, a part number somebody finally found, a rack height that was guessed - all of that gets noticed while looking at a quote. The **edit** button on a row that's priced from the catalog opens that entry, filled in, and saves it back to the catalog file, so the correction reaches every room that quotes the part rather than just this one. A row with no entry behind it still offers **add** instead. Connectors are left alone - those are edited on the **Catalog** tab.

Is it in the room config?

The estimate is where a room gets specified - parts picked with quantities and a total - and the control side is usually built weeks later. A line that never becomes a device block is a box that gets ordered, delivered, racked, and then has nothing to drive it.

So every equipment row carries a flag:

- **orange flag** - quoted, and the room config has never heard of it. The flag is also the button: *Add to the room config* creates the device block for its family with this room's defaults and the driver that claims the model. A line typed here becomes boxes on the diagram first (one per unit), because a device has to exist before it can have a block - so the cost line is replaced by the drawn devices it was quoting.
- **a tick** - it's in the config, and the tooltip names the block.
- **a box icon** - marked as a **spare**: bought for the shelf on purpose, never installed here at all. It stays on the quote and stops being flagged.
- **a broken-link icon** - marked **not part of the room config**: it *is* in the room, and the processor has no business talking to it. Available on a drawn box and on a line quoted here, because an owner-furnished display is usually quoted before anybody draws it. The building's network switch, a codec another department manages, an owner-furnished display, a passive splitter. It stays exactly as it was - drawn, selectable, cabled, racked and priced - and drops off every "missing from the config" list: the flag here, the control-side prefill, and the missing-module report. The same checkbox is on the box's own editor on **Signal Flow**, next to *Not on the cost estimate*, and the box wears a small broken-link badge on the diagram.

Those last two are the escape hatches that keep the flag meaningful. Every room has boxes whose honest answer to "why is this not in the config" is "it never will be", and a warning nobody can clear is a warning everybody learns to ignore.

A hand-typed line with no catalog part behind it can't go in - there's nothing to build a device from. Add it to the catalog first with the library button on the same row.

Base costs for cable

Cable is the hole in every early estimate: the runs are counted off the diagram exactly, then priced at nothing, because the catalog ships made-up leads for a handful of types while a real order is "a 25 ft HDMI and a 50 ft

HDMI".

The **price tag** button on a cabling row puts the shop's typical figure for that type and length on the base-cost card - `base_costs.json`, the same file the device figures live in, read by every room. Enter it once and every estimate prices that length off it. A price typed on the room still wins, and a line costed this way is marked *Base cost* and counts towards the estimate being a budget rather than a quote.

A figure entered against a type with no length ("Cable: HDMI") covers every length that hasn't got one of its own.

What the card was priced on

A base-cost figure is what a whole refresh plan is built out of, and until recently it was a number with nothing beside it saying which product, at what spec, in which year. A card can now record the **model it was benchmarked on** and the day it was set.

It doesn't change the price - the figures are still the figures. It says where they came from, so a card set on a 2022 projector in 2026 can be *seen* to be four years stale rather than having to be remembered to be, and so a finance office asked to approve eleven of something can be told what they are eleven of.

Both are written for you by the campus report's **Current models** tab, which is where a category's figure is normally decided. See *The campus*.

Swapping a unit

The wrong box usually gets noticed here - the total is what people look at - so the fix is here too. The **replace** button on the right of an equipment row picks the new product out of the catalog and puts it everywhere the room records the old one:

- **the cost line** reprices off the new catalog entry
- **the signal flow** gets the new product under every box the line counts - connectors, rack height, power and heat - and the runs already drawn move onto the matching connectors
- **the cabling schematic and cable schedule** follow, because they're built from the flow rather than stored
- **the control side** - the config block the box came from - gets the model and the Python module that claims it
- **the name**, where the name was the product: "Projector 1 - PowerLite L630U" becomes "Projector 1 - PT-MZ682BU8". Only the model part moves - "Projector 1
    - " is what the room calls that position, and the position hasn't changed - and a name that never mentioned the model is left alone

A line of quantity three swaps all three boxes and all three config blocks. Any price you had typed against the old model is cleared, because a figure negotiated for one product isn't the price of another.

It works in both directions. Picking a model on the **Devices** tab moves the same room: the drawn box becomes that product - connectors and all, where the AV catalog knows the model - the name follows, and the

estimate reprices, because the estimate counts the boxes on the diagram.

It goes through silently when there's nothing to decide. It stops to ask when there is:

- **no module claims the new model.** You can still go ahead - a part often arrives before its driver does - but the module is cleared off the config block rather than left naming a driver for the old box, and the device shows a red banner on the **Devices** tab until somebody picks one. See *When the model and the driver don't match*.
- **the module changes and this room's settings disagree with its defaults.** The same question the Devices tab asks when you pick a model there, with the same two answers: keep the room's settings (the IP address and port are facts about the install) or apply the new module's defaults.

Cancel means nothing happened - you're asked before the first write. A run the new model has no connector for is removed rather than left pointing at nothing, and the message says how many, so you can draw them again on **Signal Flow**.

The job

The Cost tab prices one room. The Project tab prices a **building** - and then follows it all the way out: who quotes it, what was ordered, what turned up, when the rest has to be bought, and whose job each piece of scope is.

A project is a list of rooms plus everything that happens around them. It doesn't copy the rooms or take them over - they stay ordinary config files you open and edit exactly as before, and the same room can sit on two projects. Press **Refresh** after fixing a price in a room and the building total catches up.

The panes

Across the top of the tab, in roughly the order the work happens:

Pane	What it answers
Rooms	Which configs are on the job, and what each comes to.
Equipment	Every part once, quantities merged, tagged to the vendor that will quote it.
Plans	The building's own drawings - floor plans, RCPs, riser diagrams - the job is quoted against.
Timeline	When each part has to be *bought*, worked back from the day the building has to be finished.
Deliveries	The purchase orders, what is on them, and what has landed.
Lifecycle	What the building already has, how old it is, and the year each position falls due.
Responsibility	Who furnishes and who installs each line of scope, room by room.
Vendors	The companies you buy from, the rules that tag parts to them, and where each quote has got to.
To do	Open questions on the job.
Notes	Signed, dated notes - who said what, and when.

Whichever pane is open, the figure in the header is what the building costs. That is deliberate: it is the number somebody came to the tab for, and it should not move when they change what they are looking at.

Getting rooms onto it

New starts a project; **Add rooms…** picks config files (several at once); **Add the open room** adds whatever you're working on. The tick beside a room decides whether it counts toward the total - untick it to price an alternate without double-counting the building. Rooms can't be added twice, because a room listed twice

doubles its cost and every one of its parts.

If a room's file has moved or been renamed, its row says so and the other rooms still price. The total is short, and the tab and the workbook both say by how many rooms.

Rooms nobody has drawn can be typed in as line items, from **Add a line item…** on this pane: a name, a date it was last done, a life, and a figure. Most of an estate has never been through this app, and a refresh plan covering only the rooms that have is a plan for a fraction of the building - short in the one direction nobody checks.

The master parts list

The **Equipment** pane is the reason this tab exists. Nine rooms with two transmitters each is eighteen transmitters - **one line**, not nine - and a vendor asked to quote one line quotes better than one asked to quote nine rooms.

Parts merge on part number where there is one, then on model, then on maker and description. Each line still says which rooms its units are for, so you can check a delivery or split a phase without unpicking the merge.

Where two rooms hold different prices for the same part - one of them has a negotiated price typed on it - the line shows the **range** rather than picking one, and the extended total is what the rooms actually pay added up, not the quantity times either price.

The chips across the top narrow it: to one vendor, to the parts nothing has claimed, to the ones with no price, to the devices nothing can drive, to spares. Treat those counts as the job's own to-do list - a building isn't ready to go out for quotes while anything is sitting in them.

Rows can be **selected in bulk** to set a lead time or a vendor across many at once, which is what turns "put six weeks on everything from that maker" into one action.

Vendors

Who sells you a part is a fact about the job, not about the part, so the tags live on the project.

Give each vendor the **manufacturers** it quotes, the **categories** it sells, or both. A new project already has the usual split - everything Extron on the direct account, and cameras, screens, mounts and USB from a reseller - so most of the list tags itself the moment you add a room.

A part is tagged by the **first** vendor whose rules claim it, and manufacturer rules are checked before category rules. That's what stops an Extron display going to the reseller who does screens. **Order matters**: drag a vendor up its list by the handle to give it priority, and the number on the row says where it sits. If two vendors claim the same rule the tab says so out loud, and names which one wins.

If you want a specific part to go somewhere else, pick the vendor on that line: it's pinned, it beats the rules, and it stays pinned even if the room is re-drawn. Pick the vendor the rules already chose and the pin is simply removed.

Anything nothing claims goes to **Untagged**.

Ticking, not typing

Both rule boxes offer **Pick from the job**: a tick-list of every manufacturer, or every category, the job actually has parts in, with a count beside each - "Display (18)" against "Screen (2)".

That matters because a rule is matched **exactly**. "Cameras" against a catalog that says "Camera", or "Extron Electronics" against one that says "Extron", is a rule that claims nothing - and there is nothing on the screen to say so. Ticking cannot go wrong that way, and the count is what makes the list readable as a decision rather than a lookup.

The typed box beside it still takes anything, because a rule for a maker or a category the job has no parts in *yet* is a real thing to want: a vendor is often set up before the rooms are drawn. Anything already written that the job has nothing for is carried at the bottom of the tick-list, ticked, under a heading that says what it is.

Category rules also match the head of a finer one, on a word boundary: a "Camera" rule claims "Camera - PTZ" and does not claim "Cameraman kit".

The count chip

"19 lines · $18,400" on a vendor card is not just a label. Press it and the **Equipment** pane opens already narrowed to exactly those nineteen parts - with their prices, their lead times and their rooms on them.

Where the quote has got to

This app builds the RFQ per vendor and hands you the spreadsheet. Everything after that - it went out on the 4th, two came back, one turned into a PO, the third has never replied - used to live in somebody's inbox, and on a six-vendor job "which of these are we still waiting on" is the most-asked question on the screen.

Four states, read off three dates:

State	What it means
Not sent	Nothing has gone out. No chip on the card - a row of gray "not sent" chips down a fresh list is noise about nothing.
RFQ sent	The request went out on a day you record. Waiting.
Quoted	A price came back.
Ordered	It went on a purchase order. The chip shows the PO number.

The chip is on the row that is **always** there, so a collapsed list of six vendors answers "who are we waiting on" without opening any of them.

Recording a quote

Quote came back… takes three things: when, for how much, and the vendor's own quote number. Those are the three needed to compare six quotes against each other and against the job's own figure without opening any of them.

The card then shows the quote **against the estimate** - "Quoted $18,400 - $1,200 OVER the job's $17,200". That gap is the whole reason a quote gets read.

Recording a quote **changes no price on the job**. The estimate stays what the parts say; this is what the vendor wants for them.

The quote PDF is attached **in the same dialog**, at the one moment somebody has it in front of them - they are reading the figure off it as they type it. Attaching it anywhere else is a second trip back to a file already closed, which is a trip that doesn't get made. Nothing is copied: the project stores where the file **is**, so moving the file moves what this opens. Any file type, not only a PDF - a scan, a screenshot of the vendor's portal, their acknowledgement as an `.msg`. What the app can draw it draws in-app; the rest it hands to the machine.

Taking the quote off the row takes the figure and the document link with it. The file on disk is never touched.

Marking it ordered

Ordered… is one action that makes three things true: it raises the purchase order, points it at this vendor, and puts **every part this vendor is quoting** onto it.

That last one is the link back from a PO number to the equipment it bought. Before it existed the first two got done and the third did not, which leaves a PO nobody can trace to any equipment and nineteen parts reading on the timeline as things nobody has bought.

The parts default to the **whole package**, because that is what a purchase order to one vendor almost always is; the count is on screen. Ordering only part of it? Do this, then untick the rest on the PO's own list on the Deliveries pane.

The signed order is attached here too, at the one moment somebody has the PDF.

Once ordered, the card offers **Equipment on PO-1188 (19)** - which opens the Equipment list narrowed to those nineteen, because that is what the sentence is asking for.

Not ordered after all stops the vendor row claiming it and leaves the paperwork exactly as it was. The PO records what *happened*; a mis-set flag on a vendor row is not a reason to unpick a job.

Purchase orders and deliveries

The **Deliveries** pane is a row per purchase order: what is on it, how much of that has landed, and the order document itself.

Each PO can carry the signed order - a PDF, a scan, an `.msg`. The same bargain the plans and the cutsheets make: the project stores where the file is, opens what it can draw in the app, and hands the rest to whatever this machine uses. Taking the link off leaves the file alone.

Deliveries are logged against a PO, or as **one-offs** for something that went on a card and never had one. A delivery says where it went: a dock, an address, or general storage. Gear **held** rather than installed is tracked as held, so "we have it, it's just not in the room yet" is a state the job can be in.

Every PO number the job mentions anywhere is one click away. The rows, the vendors, the part orders and earlier deliveries are all swept, each in the spelling it was first seen in. A number typed onto a part before anybody raised the PO, or marked on a vendor whose row was later renumbered, is still on the dropdown rather than something to read off another screen and mistype. The **Add a purchase order** box goes further and offers the numbers the job knows but has **no row for** - until a row exists there is nothing to attach the order to and nothing to tick equipment onto.

Plans

The building's own drawings - the architectural floor plan, the reflected ceiling plan, the riser diagram, the electrical sheet somebody was sent as a PDF. Not the room's floor plan, which is a picture you drag locations onto; this is the set the job is quoted **against**.

PDFs and images open in the app. Anything else opens in whatever this machine uses for it.

The delivery timeline

One date drives it - the day the building has to be finished - and one figure per part: how long that part takes to arrive. Everything on the pane is the subtraction between them, so moving the deadline moves every order date with it.

The exception is the point. A job does not need everything on the same day: screens, mounts, floor boxes and conduit go in while the walls are still open, weeks before the rack is delivered. Those parts carry their **own** need-by date, and the schedule prefers it over the project's - which is why the order-by column can show a part being bought two months before a job whose deadline is in June.

Read it as a **list of days**, not a list of parts. An order date is a trip to the purchasing office, and eleven parts sharing one is one trip; grouping them is what turns a hundred-row table into the half-dozen dates somebody actually has to put in a calendar.

Phases are what a job is really broken into. Each carries its own delivery date and the parts on it are scheduled against that, so the reading this exists for - whether the infrastructure order going in months before the tech order actually lines up with when the walls close - is on one screen.

Quote requests are on the pane too, above the orders: every vendor whose RFQ has gone anywhere, the one that needs chasing at the top. Sent before quoted before ordered, and inside each, oldest first - an RFQ sent three weeks ago and never answered is the thing to chase, and a quote that came back a month ago and hasn't turned into an order is the thing going stale. Each row says how long it has been sitting ("out 3 weeks with no answer"), what the package is worth, and offers the quote document.

What has already gone is a block of its own under that: the purchase orders, newest first, each with what it bought and how much of that is marked arrived.

The rail

Above the lists, the whole job is drawn on **one line**: today, the first order, every order date as a dot, each phase's on-site day, each vendor's quote request in that vendor's own color, and the delivery deadline.

Nothing on it is a new fact - every date is one of the cards below - which is exactly the point. A list has no distance in it: two dates a fortnight apart and two a year apart are one row apart either way. The rail is the same schedule seen from far enough away to have a shape.

It zooms. Fitted, it shows the whole job; on a three-year refresh that is about four days per pixel, which makes every date in a fortnight one dot. From there it goes in as far as thirty-two times, at which point three years of rail is about six weeks in the frame, and it scrolls sideways under a bar.

The readout between the arrows says how much of the job is in front of you - "3.0 yr", "9 mo", "6 wk" - rather than a percentage, because a stretch of time is the number somebody wants off a calendar. Zooming holds the middle of the frame rather than jumping back to the start, and the fit button brings the whole job back from wherever you had got to.

Into somebody else's calendar

The timeline is read by whoever is building the job; the purchase order is raised by somebody who lives in Outlook and will never open this app. A date that isn't in their calendar is a date that gets missed, so the schedule exports as an **ICS**: one all-day event per order date, with an alarm a week before.

The responsibility matrix

Who furnishes and who installs each line of scope, room by room. The document that settles "we thought *you* were pulling that conduit".

Rooms run down the frozen left column; scope items run across the top, each with **Furnished by** and **Installed by** under its name and, where there is one, a **cutsheet** on a row of its own that opens in the app. Under those sits the strip that carries the word ROOM, directly over the rooms it names - above it the sheet is an agreement about who does what, below it the same columns are quantities per room, and the heavy rules top and bottom are there so nobody reads a count as a commitment.

Each party carries its **own color**, the same one wherever its name appears, so one contractor's share of the sheet is visible without reading a cell. A line nobody has been named on reads **NOBODY** in the error color - a blank is the exact thing this sheet exists to catch, and the tinted chips around it would otherwise make an empty cell look like one more quiet agreement.

Hovering anything on a line lights the **whole line** right across the sheet, frozen column included. A room name on the left edge and a number twenty-eight columns to the right of it are two hand-widths apart on a laptop, and the eye tracks a moving highlight in a way it cannot track a fixed stripe.

The scope headings size themselves to the names they actually carry, so a line called "Conduit, back boxes and pull strings for every floor-mounted connectivity point" is read rather than ellipsized. The sheet zooms and fits like the year grid, and columns are re-ordered by dragging the grip in the heading - the order is content on this document, because it is grouped the way the work is sequenced.

It goes out two ways: a **spreadsheet** for the people who work from it, and a **picture** for the people who only have to see it.

Swapping a product everywhere at once

The projector everyone specified is discontinued, and nine rooms have one. Press the swap arrow on that line and pick the replacement: every one of those boxes, in every room on the project, becomes the new product.

You get a preview first. It tells you how many boxes in how many rooms, how many of the cables already drawn will carry across, how many have nowhere to go on the new box and will be **removed**, and whether the control blocks are about to lose their module. Nothing is written until you say so.

Each room moves as a whole - the drawing and the control config together, so a room never ends up commissioning the old device off a drawing of the new one. The box keeps its position, its rack slot and everything about it that belongs to the room; what changes is what the product is. Names follow: "Projector 1 - L630U" becomes "Projector 1 - PT-MZ682BU8", and a name that never mentioned the model is left alone. IP addresses, ports and control ids are kept.

Two things worth knowing before you press it:

- **There is no undo across the project.** The room you have open is undoable the usual way; the others are written straight to their files. The preview is the safety net, so read it.
- **The room you have open is not written.** It's changed in memory instead, so the editor and the file can't disagree - save that room afterwards to keep it.

If a room's file has moved or can't be read, it's named in the preview and left on the old product rather than silently skipped.

> If the reason for the swap is that the product was **discontinued**, consider naming its successor on the catalog entry as well - see *Retiring a product*. The swap changes what the rooms hold; the successor changes what every room still holding the old one is *budgeted* at, which is a different question and the one the refresh plan asks.

Devices that nothing can drive

The same check the AV and Cost reports do for one room runs across the building. If no Python module claims a device's model, it says so - and swapping onto a product whose driver doesn't exist yet is the most common way to create one, which is why the two live next to each other.

It never blocks anything. Specifying a device before its driver is written is normal. It just makes sure nobody finds out on site.

You'll see it in three places:

- on the master parts list, on the line itself, naming the rooms that still need doing - with a filter chip to show only those lines
- in the project workbook, as a **Control Gaps** sheet listing every one of them by room, with a short "what needs doing" summary that splits them into pick the module / write the driver / choose a model / draw the device

- in the warning count at the top of the tab and in the workbook's "check before this goes out" block

Cable and rack hardware are never flagged - they were never going to have a driver. Neither is anything you've marked as never controlled on the Catalog tab.

"Never needs one"

Two different things land on that list. One is a driver nobody has written yet, which somebody will get to. The other is a thing that simply has no control interface - a passive splitter, a plate, a USB capture stick - and no amount of work will ever change that.

Leave the second kind on the list and it just grows, until the list is mostly noise and the real ones get skipped. So you can retire it: press **Never needs one** on the row, right next to where it's complaining, and confirm.

That's saved to the **catalog**, not to this project. Every room in every job that draws one stops asking about it from then on - which is what you want, because it's a fact about the product.

The same choice is on the Cost tab, in the little flag menu on the row: **"This product never needs a module"**.

Be careful which of the two you want, because they look alike and aren't:

- **"Not part of the room config"** - this box, in this room, isn't yours to drive. An owner-furnished display, the building's switch, another department's codec. The same product next door might well be driven.
- **"This product never needs a module"** - nothing can drive one of these, anywhere, ever.

The product has to be in the catalog already. If it isn't, use **Add to catalog** first - the app won't invent an entry from a quote line, because one built that way would have a name and nothing else and would get in the way of the real entry later.

Changed your mind? Untick **Never in the room config** on the Catalog tab. (The Project tab's button only goes one way, because once a product is marked it stops appearing on that list - there'd be no row left to press.)

To do, notes and history

To do holds the open questions on the job. **Notes** are signed and dated for you, so who said what and when survives the person who said it moving on.

Every edit to the job is logged under the thing it was about. "This says bought

- who said so, and when" is a question about the **part**, and a single line on

the vendor cannot answer it, so orders, quotes, prices and dates are all recorded against both.

Working through the rooms

Once the project has rooms, there's a **room picker in the title bar** - on every tab, not just this one. Pick a room, or use the arrows to step through them, and the editor loads it. Everything: the config, the drawing, the racks, the estimate. Then go to whichever tab you want and you're looking at that room's version of it.

That's the point of putting it in the title bar rather than here. The tab is the *question* and the room is the *subject*, and you should be able to change one without losing the other. Sitting on Cost and stepping through eight rooms is a perfectly ordinary thing to want to do.

Your edits show up in the project straight away. Type a price on the Cost tab and the building total moves - you don't have to save first and press Refresh. The project reads the room you're editing out of memory, and every other room off disk.

The catch, and the app says it in two places: what you've changed isn't in the room's *file* yet. The Rooms list marks the open room and says when it's being counted with unsaved changes, and a **Save room** button appears in the title bar whenever there's something to save. That button writes straight back over the room's own file - no dialog, no picking a folder.

Switch rooms with unsaved work and you'll be asked whether to save first, because switching reads the next room off disk and anything not saved would go.

Getting it out

Workbook writes one spreadsheet with everything: what the building costs, each room's share, the master parts list, a tab per vendor, the quote requests, the orders, the schedule and the plan, and a tab per room.

Quote requests writes one spreadsheet **per vendor** into a folder you pick. That's the file you email. It has that vendor's parts, the rooms they're for, your own estimate to compare a returned quote against, and empty columns for their price and lead time - and nothing else. No labor, no tax, no project total, no other vendor's parts. Send the whole workbook instead and you've sent a supplier your margins and a competitor's pricing.

The refresh plan

Everything else in this app answers "what is going **in**". This answers the question that comes next: what is already there, how old it is, and which year it has to be replaced.

The **Lifecycle** tab does it for one room; the Project tab's **Lifecycle** pane does it for a building; the campus report does it for an estate. All three are the same arithmetic over the same data, so a year on one can never disagree with a year on another.

How the years are worked out

Every position on the drawing carries the day it went in. The install year is **year one**.

The bands are the RYG sheet's bands: on an eight-year cycle, green for years one to five and amber for six to eight, then red. Amber is **graded** rather than flat, because three years of it are not one state - the screen paints a six-step ramp and a printed sheet carries the same six codes, so a mono copy has not lost the half of it that says which of the three amber years a room is in.

The life a position is held to comes from three places, most specific first:

1. **the position itself** - somebody saying "this one, sooner"
2. **the catalog entry** for its model - what the product does in general
3. **the blanket cycle** for anything nobody has recorded one for

Each row says which of the three it used, so a plan can be argued with rather than only believed.

A position with no install date reads **UNKNOWN** and says so - a room whose dates nobody has collected is a room nobody can plan, and costing it at nothing would hide that. Jack fields and patch panels are never on the cycle at all; mounts, poles and frames can be taken off it by hand, and stay tracked, listed under a heading that says why they are not on the plan.

What a replacement costs

The catalog's price for the model, and failing that the **base cost card's** figure for its category. A figure off the card is marked as a *typical* price rather than quietly mixed in with the ones that are the box's own - a typical price presented as a quote is how a budget goes wrong quietly.

A retired model is priced at what replaces it. The drawing says what is in the room; this figure is what it costs to take that out and put something in, and for a discontinued product those are two different numbers - often a third apart, always in the same direction. When the catalog entry names a successor, that successor's price is what the plan uses, and the row says whose price it is.

The chain is followed all the way: a 2012 model replaced by a 2016 one replaced by a 2024 one prices at the **2024** one, because that is what the purchase order would say. A retired entry that names nothing keeps its

own price, because that is still the best figure anybody has, and a name the catalog no longer has stops the chain rather than breaking it.

The year grid

A room per row, a year per column, and what falls due in each. This is the sheet a capital budget is written off.

A room is rarely one date - the projector went in in 2016 and the displays in 2019 - so a room appears in **every** year it owes something in rather than only in the first. The sheet zooms and fits, because it is read both for a figure and for its shape.

The line under the grid, and what to set aside

The grid says which year and how much. It cannot show a **shape** - forty rows of cells have to be read across before the one bad year in them is visible - so the same figures are drawn as a line under it, on all three screens: a room, a building and the estate. Hover any point and it names what is in that year.

The dashed line across it is the other half of the question. A refresh plan is a demand curve, and a demand curve with one bad year in it is the reason phasing exists, so the chart also draws what the **same plan** would cost every year if the peaks were pushed off and the whole of it spread evenly from today to the end of the plan. The gap between the spike and the flat line is the size of the deferral being argued about.

That flat figure is printed beside the totals as **to set aside each year**. It is the one number on the sheet somebody writes down: not "what does the refresh cost" but "what should we be saving". It is worked out over the years still **ahead** - dividing a plan by twenty years when twelve of them are already gone produces a comfortable figure nobody can spend to.

Both the line and the figure are on the picture, so a sheet handed to a budget meeting carries the shape and the ask rather than a grid of cells somebody has to add up.

Modelling a different cycle

"What if we did every room at ten years" is the question every capital planning meeting asks, and answering it by editing the life on forty rooms is not a what-if - it is a rewrite with no way back.

So the plan is **restated** instead. The **Cycle** picker on the sheet's own header row rebuilds every grid, chart, picture and spreadsheet as though everything were on the cycle you asked for, whatever life is recorded against it. It offers six common cycles and **Type a year...**, which takes any number of years up to 60 - a department funding a nine-year rotation asks for nine rather than picking the nearest entry on somebody else's list.

One setting, three screens: a room, its building and the estate are the same question at three sizes, so a cycle picked on one is the cycle the others read at.

Nothing is written anywhere. A note above the sheet says what is being assumed and what it moved, anything exported while it is on says so on its face, and **As recorded** puts the plan back in one press. One thing is never restated: a position somebody has taken off the refresh cycle stays off it - that is a statement that the bracket

is not on the plan at all, not a life figure to be argued with.

The campus

Several buildings on one sheet. Add the project files, or point it at a folder, and every figure is read off the jobs themselves in one pass over disk - so nothing on the sheet can be older than anything else on it.

A building's figure on the calendar is a **way into** that building's own plan. The campus total row is not, because it is not any one building.

The assembly is a document in its own right: it can be named, saved, reopened, and undone sixty steps deep like everything else in the app. It goes out as a picture, a spreadsheet or an online copy.

Current models

A whole refresh plan is built out of **one number per kind of thing** - what a projector costs, what a switcher costs, what an interface costs - and those numbers had no provenance at all. Somebody typed them onto the base cost card once, and an estate was budgeted off them for as long as nobody re-typed them.

Two ways that goes wrong, both quietly. The number **goes stale**: it was right in 2022, nobody re-typed it, and the 2026 plan is short by four years of price rises across every room. And the number **cannot be argued with**: a finance office asked to approve eleven projectors wants to know what they are eleven *of*, and "about $4,200" is not a specification.

The **Current models** tab beside the plan is where that number gets decided, in front of the evidence. For every kind of thing the estate actually holds it says:

- how many positions there are
- which models they are, commonest first, and how many of those the catalog has already **retired**
- what the plan presently budgets them at - added off the plan's own rows, so this figure and the year grid can never disagree
- what it is currently **benchmarked on**, and how long ago that was set

Then pick this year's model out of the catalog. The comparison lands **before** anything is committed: the unit price, forty-one of them, and the gap against what the plan assumes. That gap is the reading - a budget short by it is a budget that fails at purchase order time.

Make this the recommended cost writes the figure, the model and the date onto the base cost card. That is the same card the room cost page, the project report and the campus report all already price from, so the decision reaches every one of them without any of them knowing this tab exists - and a card set on a 2022 projector in 2026 can then be *seen* to be four years stale rather than having to be remembered to be.

Both published tiers are written, off the catalog entry's own two figures. A card with one price on it has every estimate at the other tier reading high or low depending on which way the job went.

Categories the estate holds nothing in are not listed. The base card ships with every family the app knows and most estates use a third of them; a tab listing the other two thirds buries the answer in blanks.

The catalog

The catalog is the department's price list and connector reference: per model, what sockets it has, how many rack units, what it draws, what it costs, and how long it lasts.

Searching ignores spaces, dashes and case on both sides, so "dtpcross108" finds "DTP CrossPoint 108". Type more than one word and every word has to land somewhere on the entry - the model, the maker, the part number or the category - in any order: "Epson PowerLite" finds the PowerLite projectors, even though no single field holds those two words next to each other. Adding words still narrows the list.

The room config knows a device's IP address; it never knows that the box has four HDMI inputs, is 2U, draws 90 W and lists at $8,500. That's what the catalog is for, and it's why one shared copy is worth keeping.

Merge exists because two engineers keep two copies and neither is authoritative: one has priced the switchers, the other has drawn their connectors. Point it at their file and every difference comes up with its own checkbox.

What an entry carries

Beyond the obvious - model, maker, part number, category, both published prices, connectors, rack units - three fields are worth knowing about because other parts of the app read them and nothing else will fill them in:

- **Clearance above / below.** Rails this model wants kept *empty* - the amp that vents upwards, the drawer whose lid opens. The rack shades them and still lets you drop something there.
- **Life (years).** How long the product lasts before it wants replacing. The refresh plan prefers this over its blanket cycle, so a catalog with lives on it produces a plan nobody has to hand-correct.
- **Never in the room config.** Nothing can drive one of these, anywhere. See *"Never needs one"*.

Retiring a product

Retired / discontinued keeps the entry out of the pickers that specify new work, and keeps everything it knows - ports, price, rack height - for the rooms that already have one. Deleting it instead would silently strip the connectors and the price from every room that used it.

On its own that answers "do not specify this any more" and leaves "what does it cost to replace the forty already on the estate" answered with a discontinued product's list price. It is a real figure, for a real catalog entry, that nobody can buy anything at - and nothing on any screen said so.

So a retired entry can name what **replaces** it. Tick Retired and a **Replaced by…** picker appears; everything that asks what it costs to replace a position holding one then asks the successor instead. The room's cost page, the project report and the campus report all price a replacement through the same ladder, so naming it once fixes all three.

The successor is **picked from the catalog** rather than typed, for the same reason every model reference in this app is: a name the catalog doesn't have is a chain that stops silently at the price it was trying to get away from. The line under the picker says what a room holding one is now budgeted at, and says so when the chain runs on - "budgeted at PowerLite L775U - the L730U is retired too, and the chain runs on to it".

Clearing it puts those rooms back on the retired entry's own price.

Getting work out of the app

Most panes export what they show, and nearly always in **two shapes for two audiences**: a *spreadsheet* for the people who work from it - a contractor prices a total, and a price gets typed into a spreadsheet - and a *picture* for the people who only have to see it, which goes in a submittal or an email to a dean.

Pictures are produced from a **preview you look at first** rather than captured off the working pane. What is on a pane is an editor, with buttons on every row, and photographing an editor produces a picture with delete buttons in it.

Save All writes the whole room into one folder: every drawing as a PNG, every report, the workbook, and the config itself. It walks the drawing tabs to capture them, then puts you back where you started.

The room workbook is one `.xlsx` with a sheet per document - devices, control, AV, racks, cabling, cost - for handing to somebody who doesn't have the app.

Per-tab exports are on every tab's own menu: this page's tables as `.xlsx`, as plain text, or straight to the clipboard.

Screenshot & annotate grabs the current tab, lets you draw on it, and copies or saves it - for the email that starts "the third output is wrong".

Online copy is on the Project tab, beside Workbook. Point it once at a folder OneDrive or Google Drive already syncs and it writes the project workbook there - Excel Online opens it as a spreadsheet, Google Drive opens it as a Sheet - so the people who ask about the job can read it without the app and without you emailing anything. Tick the box and the project file goes with it, which makes the folder enough to *open* the job on another machine rather than only to read it.

It rewrites the same file name every time, so a share link you send once keeps opening the current figures. The box says when it last went out, because a copy three weeks behind is worse than none - whoever is reading it has no way to tell. Tick *Update it every time the project is saved* and it never gets behind.

What comes back. Two sheets in the published workbook are a form rather than a report: *Deliveries (edit)* and *Purchase orders (edit)*. Anybody the folder is shared with can type in them - a delivery that landed, a quantity that was wrong, the room a pallet went into - and *Pull updates* reads them back. Leave the Row id alone; add a line with a blank Row id to log something new. Dates as `2026-04-20`. Every change is listed for you to check before a word of it is written, each one lands in the job's history saying it came from the online copy, and nothing is ever deleted by an import: a row missing from the sheet is one somebody filtered or never scrolled to. Every other sheet is a picture of the job and is overwritten on the next publish.

The campus and single rooms publish the same way, into the same folder: the campus screen's *Hand over* menu has an *Online copy* item, and the cloud button in the toolbar publishes the open room (or asks, when a room is open inside a job). Everything lands beside everything else under names that sort - `Chico_campus.xlsx`, `Bessey_Hall_project.xlsx`, `BSS_103_room.xlsx` - each with its `.json` next to it. Nothing in that folder needs this app to open: it is spreadsheets and plain JSON, readable and editable with or

without it.

The index ties them together. Every publish also writes `index.xlsx` and `index.json`: one row per document, saying what it is, what it belongs to, what it holds, when it last went out and which files are its own - campus first, then the jobs on it, then the rooms in those. It accumulates, so a campus published in March is still on it in June with the date that says how old it is. The most useful table on it is the second one: what a job or a campus names that is *not* in the folder, so nobody hunts for a file that was never published. Delete either index file whenever you like; the next publish writes it again.

Help, in the app

This guide is a file somebody has to find, open in another window and search separately - which on a laptop with a drawing open is a thing that does not get done. So the same material is **in the app**, on the question mark beside the gear in the title bar, or on **F1** from anywhere.

It opens on its search box, because nobody opens help to browse a contents page: they open it because a control did something they did not expect, and the words they have are the words off that control.

The search runs over **everything** - the title of each feature, the part of the app it belongs to, where to find it, its prose, and the words somebody would actually type when they don't know what it's called. "RFQ" reaches the quote requests, "RYG" reaches the refresh plan, "MSRP" reaches the pricing tier, "BOM" reaches the master parts list, "who installs it" reaches the responsibility matrix.

Every word you type has to appear somewhere, so a phrase narrows fast rather than widening. A hit in the name of a feature outranks a hit buried in its prose, and a whole word beats a fragment of one - typing "rack" finds *Rack elevations* rather than *Phases and tracks*.

Each page opens with **where the thing is**, as a path through the app rather than a position on a screen, then what it does and why it works that way. The text is selectable, because half of what help gets used for is pasting a sentence into an email to whoever asked. The keyword chips at the foot of a page are a press each, which is how somebody gets from one feature to its neighbors without knowing what they are called.

It opens over whatever you were doing and closes again without losing it.

The Flow Rules builder

What a rule is

Drawing a room takes two kinds of knowledge, and only one of them belongs to the program.

The mechanical half is the app's: which socket the number `3B` names on a DTP CrossPoint, whether a connector already has a lead on it, how to split a run into two cables and a box. You never have to think about that.

The other half is a set of decisions *your shop* has made:

- `input_pc` means the room PC, and the room PC is a box with an HDMI out and a USB in
- a twisted-pair output reaching an HDMI display needs a DTP 4K 230 receiver between them
- the Toggle's DEVICE ports carry the DSP, the AV Bridge and the doc cam, in that order, and HOST 1 is the PC
- an outlet labeled "Switch" is the matrix, not the USB switcher

Every one of those used to be compiled into the program. Now they're rules in `av_flow_rules.json`, and the **Flow Rules** tab is where you read and change them.

> With no rule file at all, the app uses its built-in rules - the same ones that used to be in the code. Nothing you have to do; the tab is there for when the defaults are wrong for a room, a building or a whole campus.

Finding your way around the tab

The list on the left is the rule families, with a count each. Pick one and the right-hand side shows what's in it, one card per rule, with an **Add** button above and edit and delete buttons on each card.

Along the top: **Save** writes the file, **Reload from file** throws your edits away and re-reads it, and **Reset to built-in** puts back exactly what the app ships with. Next to the title you'll see either the file the rules came from or *"Edited - not saved yet"*.

Edits take effect immediately, before you save. The file on disk is only touched by Save, so you can try a rule, look at a room, and decide.

> To see a rule change on a room that's already drawn, go to the **AV Flow** tab and press **Recreate from config**. The drawing is rebuilt from the config and the rules as they now stand.

Naming a box in a rule

Several rules have to point at "whatever box in this room is the DSP". One small syntax does that everywhere:

You type	It means
`DSPDEVICE_`	the family - the first numbered block of it that fits
`RECORDERDEVICE_1`	exactly that config block
`input_doc_cam`	the box the source rules place for that config key
`AV Bridge 2x1`	a catalog model, matched against what's on the canvas
`RECORDERDEVICE_|MEDIAPORTDEVICE_`	try the first, then the second

The family form is the one you want most of the time. "The DSP" means the DSP this room has, whether it's block 1 or block 3, and the app skips blocks that can't do the job - a DSP with no USB socket isn't the DSP the USB rule means.

The families, one by one

Source boxes

An `input_` key whose box has no config block of its own: the room PC, the doc cam, the laptop at a plate, a DVD player. The config says which switcher input it's on; this rule says what the box actually is.

Each one carries a name for the drawing, a catalog model (where its connectors, price and rack height come from), where in the room it lives, and whether the room is paying for it - the presenter's own laptop belongs on the drawing but not on the quote.

Source devices

An `input_` key naming a device the config already describes, so the box is already on the canvas and only the cable is missing: `input_wireless` is the wireless block, `input_inst_cam` is camera 1.

Display outputs

The other end of the matrix: `output_proj_1` feeds `PROJECTORDEVICE_1`. If a room has a display output for a display it doesn't have, you get a plain-English finding rather than a mystery gap.

Destination boxes

Same idea as source boxes, at the other end: a confidence monitor, the assisted-listening transmitter. These carry one extra setting - which connectors the lead may land on. Assisted listening is line audio, and looking for a video socket on it finds nothing, which is why the rule says so rather than the program special-casing the key by name.

Capture

Where the capture feed lands. A room's capture box is a MediaPort in one build and an AV Bridge in another, so the rule names the alternatives in the order they should be looked for.

Extenders

The most useful family, and the one worth understanding.

A rule here says: *when a run leaves the switcher on one kind of connector and arrives at the far end on another, this is the box that goes between them.* The run becomes two cables and a box on the drawing, and the box lands on the estimate.

The shipped rules are the two directions of the same fact:

At the switcher	At the far end	Which end	Box
`hdbaset`	`hdmi`	an output	DTP HDMI 4K 230 Rx
`hdbaset`	`hdmi`	an input	DTP HDMI 4K 230 Tx

Before it invents anything, the app checks whether the far end takes twisted pair itself - a projector with an HDBaseT socket needs no receiver, and putting one in quotes a box the room doesn't need.

USB switchers

One line per port, in port order: the first line is DEVICE 1, the second DEVICE 2, and so on, and the same for the HOST ports. Leave a line blank to leave that port empty.

The shipped rule for a Toggle reads:

```
The USB switcher:  USBDEVICE_1

DEVICE ports       DSPDEVICE_
                   RECORDERDEVICE_|MEDIAPORTDEVICE_
                   input_doc_cam

HOST ports         input_pc
```

Expansion bus

The words that identify an expansion-bus connector on a box - `EXP` and `EXPANSION` as shipped. Matched as whole words, so `EXP` doesn't also match `EXPO`. If a manufacturer spells theirs differently, add the word.

Outlet names

Outlet labels that name a device outright, whatever else on the canvas answers to the word. "Switch" is the one that needs saying: in a room built out of Extron gear it means the matrix, but scored on the word alone it ties with the USB switcher and every "Switcher 2" - and a tie draws nothing at all.

Matched on the whole label, so this settles "Switch" and leaves "USB Switch" alone.

Five things you might actually want to do

The room has a kind of source the app doesn't know

Say your template gained `input_cable_box`.

1. Go to **Source boxes** and press **Add**.
2. Config key: `input_cable_box`.
3. Name on the drawing: `Cable TV box`. Catalog model: the model from the Catalog tab. Where it lives: `rack`.
4. Save the rule, then **Recreate from config** on the AV Flow tab.

The box is placed and cabled to whatever input the config gives it. If the catalog doesn't carry that model yet, the card tells you so - the box still draws, with the generic family connectors, and costs nothing on the estimate until you add it.

We use a different receiver

Open **Extenders**, edit the receiver rule, and change the model to the one you actually buy - `DTP HDMI 4K 330 Rx` for the long runs, say. Every room you redraw from then on puts that box in, on the drawing and on the quote.

Our Toggles are wired the other way round

Open **USB switchers**, edit the rule, and reorder the DEVICE port lines. If your doc cam is on DEVICE 2 and the AV Bridge on DEVICE 3, swap those two lines. That's the whole change.

This building's capture box is a MediaPort

Nothing to do - the shipped capture rule already tries a MediaPort, then a recorder, then a USB switcher. If you have a fourth kind of box, add it to the end of the list with a `|`.

An outlet label your team uses

Open **Outlet names** and add it: the whole label on the left, the device family it means on the right. Now that outlet gets a mains lead drawn to the right box instead of being reported as a tie.

When a rule doesn't fire

- **The card shows a red line about the catalog.** The model named in the rule isn't in `av_devices.json`. Not fatal - the box draws with the generic connectors for its family - but it's nearly always a typo, and it will cost nothing on the estimate.
- **Nothing is drawn for a key.** Check the config actually has that key with a value. A blank or `none` means "there is no cable", and that's respected.

- **A box you named isn't found.** Family prefixes need the trailing underscore (`DSPDEVICE_`, not `DSPDEVICE`). A model name has to match the catalog spelling.
- **The drawing didn't change.** Press **Recreate from config** on the AV Flow tab. Rules apply to what gets drawn *next*; they don't rewrite leads already on the canvas.
- **You want the old behavior back.** *Reset to built-in* returns every family to the shipped rules. Nothing is written until you press Save.

The Schema builder

What the schema decides

Open the **Devices** or **System** tab and every field you see - its label, the description behind the info button, whether it's a switch or a dropdown, what that dropdown offers, whether it appears at all - comes from `ui_schema.json`. So does the list of device families the Wizard manages, and what a new or newly loaded room is given when it's missing something.

That file has always been editable. What was missing was a way to edit it *here*, against a real config, instead of in a text editor with a second window open to see which keys you'd covered.

Coverage: the part you'll use most

Coverage is that second window, built in.

Pick a block of your default config file - `SYSTEM_SETUP`, `PROJECTORDEVICE_1`, whatever the template has - and every key in it is listed against the schema entry that describes it, with a count at the top: *"38 of 120 keys described"*. Turn on **Not described yet** and you're looking at the list of fields that currently show up as a raw key with a plain text box.

Press **Describe** beside one and the field editor opens with the key already filled in and its type guessed from what the file holds - a `true` becomes a switch, a number becomes a numeric field. Give it a label and a description, press Save, and go and look at the System tab: it's already there.

Another config file points Coverage at a different config, which is how you check the schema against a real room rather than the template.

Describing a field

The field editor holds everything a schema entry can say:

Setting	What it does
Config key	The key it describes. May contain a `*` to cover a family: `power1_outlet_*`.
Rendered as	The control: `auto`, `text`, `int`, `double`, `bool`, `dropdown`, `combo`, `hidden`, `room_sources`, `module_states`.
Label	What the field is called on the tab.
Description	The text behind the info button.
Helper line	Small gray text under the field.
Options	For a dropdown: one per line, `value | label`. The label is optional.

Setting	What it does
Keys this one field writes	For a combo - one dropdown that sets several keys at once.
Command in the module	For `module_states`: the entry in the driver's `self.Commands` whose states fill the dropdown.
Hide when	Conditions that make the key irrelevant. Any one true and the field isn't drawn, isn't added to new devices, and isn't offered by Check Defaults.
Label when	A different label under a condition, first match wins.
Show even when the block has no such key	The field appears anyway, and the first edit writes it. This is how a new setting reaches rooms built before it existed.

Conditions are written the same way everywhere: `key=value`, `key!=value`, or `key~text` for "contains". All case-insensitive.

The other sections

Fields is the plain list of everything the document describes globally, with a search box - the same editor, reached from the other direction.

Device fields are entries scoped to one family or section, keyed by a pattern like `PROJECTORDEVICE_*`. A scoped entry wins over a global one on those tabs, which is how a projector's `input` can be a dropdown filled from its own driver while `input` elsewhere stays plain text.

Device families is the Devices tab itself: each family is a `dev_` count key, the section prefix its blocks use, a label, the highest count the Wizard offers, and any `SYSTEM_SETUP` keys the family owns (setting the count to 0 removes those, because outlet names with no power controller behind them are dead data). Add a family here and it gets Wizard counts, device tabs, pruning, audits and a `"0"` default with no code change at all.

> Editing any family writes the *whole* list into the file. That's deliberate: defining families at all replaces the built-in list, so a half-list would silently drop the rest. The tab tells you when it's about to do this.

Defaults covers the three kinds:

- `SYSTEM_SETUP` defaults, added to a loaded room that's missing them
- whole **section blocks** a room is given when it has none (a metrics block, say)
- **device defaults**, merged into every new block of a family - projectors get their `input` and `relay_host`, DSPs get their audio group numbers

Each is edited as `key = value` lines, one per line, with JSON values so `true`, `3` and `"text"` all keep their type.

Consistency is the cross-check: *when this is true, that must be true too*. A violation never blocks an edit - it paints the red mismatch outline on the fields the rule names, with your message as the red helper line. `{key}`

in the message is replaced with that key's live value.

Raw JSON is the whole document in a text box, validated before it's applied. Anything the forms don't cover lives here, and a document that won't parse changes nothing.

Saving, and what's kept

Save writes the document that was **read**, with your edits in it - not a regenerated approximation. Two consequences worth having:

- the `__comment` entries the file uses to explain itself survive
- so does anything a later build understands and this one doesn't

If nothing has ever been read - you're running on the app's built-in schema - Save refuses, rather than replacing a schema somebody has with an empty document. Point App Config at a `ui_schema.json`, or start one from Raw JSON.

Four things you might actually want to do

The template gained a key and the field looks raw

Coverage → the block → **Not described yet** → **Describe**. Label, description, type. Save.

Make a text box into a dropdown

Edit the field, set **Rendered as** to `dropdown`, and put the options in one per line. Use `value | label` when the stored value isn't what you want people to read:

```
Yes | Yes (single mic with ducking)
No  | No (single mic with mute)
Ceiling | Ceiling (voicelift and mute)
```

Hide a field that doesn't apply

A serial port means nothing on a device connected over the network. Put `com_type=Network` in **Hide when** and it stops being drawn, stops being added to new devices, and stops being offered by Check Defaults - and the keys that just became irrelevant are removed when you change the gating value.

Add a whole new device family

Device families → **Add family**. Count key `dev_lifts`, prefix `LIFTDEVICE_`, label `Projector Lifts`. Save, then look at the Wizard: it has a count dropdown for lifts, and setting it to 2 creates the blocks and their tabs. Give the family its fields under **Device fields** with the pattern `LIFTDEVICE_*`, and its baseline values under **Defaults**.

Reference: ui_schema.json

Everything here can be done on the **Schema** tab. This chapter is for when you would rather look at the file - reviewing a change, or diffing two copies.

The shape of the file

One JSON object. `fields` is the only part it must have; everything else is optional, and a part you leave out keeps the app's built-in behavior.

```
{
  "schema_version": 1,
  "__readme": [ "keys starting with __ are ignored, so notes live here" ],

  "fields":            { "com_type": { ... } },
  "device_fields":     { "PROJECTORDEVICE_*": { "input": { ... } } },
  "section_fields":    { "METRICS_CONFIG":    { "enabled": { ... } } },
  "device_defaults":   { "DSPDEVICE_*": { "group_prog_gain": "1" } },
  "device_types":      { "dev_projectors": { "prefix": "PROJECTORDEVICE_" } },
  "system_defaults":   { "gui_mic_mix": "No" },
  "section_defaults":  { "METRICS_CONFIG": { "enabled": false } },
  "consistency":       [ { "when": "...", "expect": "..." } ],
  "runtime_written":   [ "SYSTEM_SETUP.last_boot" ]
}
```

A field entry

```
"com_type": {
  "type": "dropdown",
  "label": "Connection Type",
  "description": "Text behind the (i) info button.",
  "helperText": "Small gray hint under the field.",
  "options": ["Serial", "SerialOverEthernet", "Network"],
  "hideWhen": ["dev_relay_power=No"],
  "labelWhen": { "gui_usb_or_vga=VGA": "VGA over USB" },
  "addIfMissing": true
}
```

Property	Means
`type`	Which control to draw. See the table below.
`label`	The field's name on the tab. Falls back to the raw key.

Property	Means
`description`	The info button's text. Falls back to the app's built-in dictionary.
`helperText`	Gray hint under the field.
`options`	Dropdown or combo choices.
`writes`	Combo only: the keys this one dropdown sets.
`moduleCommand`	`module_states` only: which command's states to offer.
`hideWhen`	Conditions that make the key irrelevant to its block.
`labelWhen`	Condition to label, first match wins.
`addIfMissing`	Draw it even when the block has no such key yet.

Types

Type	Draws
`auto`	Inferred from the value: bool to a switch, int to a numeric field, anything else to text.
`text`, `int`, `double`	A text field that stores that type.
`bool`	An on/off switch.
`dropdown`	A fixed list of options.
`combo`	One dropdown that writes several keys together.
`hidden`	Never drawn - for keys a combo or the Wizard manages.
`room_sources`	A dropdown of the sources *this* room has, read off its `input_*` keys.
`module_states`	A dropdown filled from the device's own Python module.

Options, two ways

Plain strings when the stored value is what you want to read:

```
"options": ["TCP", "SSH", "UDP"]
```

Objects when it isn't:

```
"options": [
  { "value": "Yes",     "label": "Yes (single mic with ducking)" },
  { "value": "No",      "label": "No (single mic with mute)" },
  { "value": "Ceiling", "label": "Ceiling (voicelift and mute)" }
```

```
]
```

Combos

One dropdown, several keys. Name the keys in `writes` and give each option a `values` array in the same order:

```
"gui_inputs": {
  "type": "combo",
  "label": "Sources & Routing",
  "writes": ["gui_inputs", "gui_tab_type"],
  "options": [
    { "label": "PC + HDMI + Doc Cam", "values": ["3", "DOC_USB_WL"] },
    { "label": "PC + HDMI",           "values": ["2", "USB_WL"] }
  ]
}
```

Wildcards

A `*` in the key covers a family: `power1_outlet_*` describes every outlet name in one entry. An exact key always wins over a wildcard.

Device families

```
"device_types": {
  "dev_projectors": { "prefix": "PROJECTORDEVICE_", "label": "Projectors / Displays" },
  "dev_lifts":      { "prefix": "LIFTDEVICE_", "label": "Projector Lifts", "max": 4 },
  "dev_power_controllers": {
    "prefix": "POWERDEVICE_",
    "label": "Power Controllers",
    "systemKeys": ["power*_outlet_*"]
  }
}
```

`prefix` is required; everything else is optional. `max` is the highest count the Wizard offers (8 by default), `keepAlivePreference` is the command names tried in order when a new device picks a keep-alive off its module, `template` is a whole block to use for new devices when the config has no block 1 to copy, and `systemKeys` are the `SYSTEM_SETUP` keys that mean nothing without this hardware - setting the family's count to 0 removes them.

> Defining `device_types` at all replaces the built-in list. Keep every family you want, in the order you want them in the Wizard. The Schema tab does this for you.

Defaults

- `system_defaults` - `SYSTEM_SETUP` values injected into a loaded room that is missing them. Defining any replaces the built-in set. The `dev_` counts are not listed here; they always come from `device_types` so the two can't disagree.
- `section_defaults` - whole blocks a room is given when it has none.
- `device_defaults` - merged into every newly created device block. Existing values are never overwritten, and an exact section name beats a wildcard.

Consistency rules

```
"consistency": [
  {
    "section": "SYSTEM_SETUP",
    "when":   "gui_tab_type~VGA",
    "expect": "gui_usb_or_vga=VGA",
    "message": "This tab type shows a VGA source, but gui_usb_or_vga is {gui_usb_or_vga}.",
    "flag": ["gui_tab_type", "gui_usb_or_vga"]
  }
]
```

Runtime-written keys

`runtime_written` lists `SECTION` or `SECTION.key` entries (with `*` wildcards) that the *processor* writes while the room runs. They stop a downloaded config being flagged for carrying things the template never had.

Reference: av_flow_rules.json

Everything here is edited on the **Flow Rules** tab. A family the file leaves out keeps the app's built-in rules; a family it defines replaces them outright.

```
{
  "sourceBoxes": {
    "input_pc":       { "label": "Room PC", "model": "PC" },
    "input_doc_cam":  { "label": "Document camera", "model": "Document Camera" },
    "input_hdmi":     { "label": "Laptop at the HDMI plate", "model": "HDMI Laptop",
                        "excludeFromCost": true },
    "input_dvd":      { "label": "DVD player", "model": "DVD Player", "zone": "rack" }
  },

  "sourceDevices": {
    "input_wireless": "WIRELESSDEVICE_1",
    "input_inst_cam": "CAMERADEVICE_1"
  },

  "destinationDevices": {
    "output_proj_1": "PROJECTORDEVICE_1"
  },

  "destinationBoxes": {
    "output_monitor_1": { "label": "Confidence monitor", "model": "Confidence Monitor" },
    "output_audio_ald": { "label": "Assisted listening", "model": "Assisted Listening",
                          "zone": "rack", "signals": "lineAudio" }
  },

  "captureDestinations": {
    "output_cc": "MEDIAPORTDEVICE_1|RECORDERDEVICE_1|USBDEVICE_1"
  },

  "extenders": {
    "rx": { "switcherSignal": "hdbaset", "farSignal": "hdmi", "onOutput": true,
            "model": "DTP HDMI 4K 230 Rx", "label": "Room-end DTP receiver",
            "zone": "wall" },
    "tx": { "switcherSignal": "hdbaset", "farSignal": "hdmi", "onOutput": false,
            "model": "DTP HDMI 4K 230 Tx", "label": "DTP transmitter",
            "zone": "lectern" }
  },

  "usbSwitchers": [
```

```
    { "switcher": "USBDEVICE_1",
      "devicePorts": ["DSPDEVICE_", "RECORDERDEVICE_|MEDIAPORTDEVICE_", "input_doc_cam"],
      "hostPorts":   ["input_pc"] }
  ],

  "expansionKeywords": ["EXP", "EXPANSION"],
  "outletAliases":     { "switch": "SWITCHERDEVICE_" }
}
```

Field	Means
`label`	What the box is called on the drawing.
`model`	Catalog model - connectors, rack units, price.
`zone`	`lectern`, `rack` or `wall`.
`excludeFromCost`	Draw it, don't quote it.
`signals`	Which connectors a lead may land on: `video`, `lineAudio`, `speaker`, `usb`.
`switcherSignal` / `farSignal`	The two ends that don't match.
`onOutput`	`true` for a receiver at a display, `false` for a transmitter at a source.
`devicePorts` / `hostPorts`	In port order. An empty string leaves that port free.

The laptop plate is one config key with two meanings, so the rule book carries both boxes: `input_usb` for a USB-C room and `input_usb (VGA room)` for a VGA one. `gui_usb_or_vga` picks between them.

Reference: key_map.json

The key map translates old files. It runs at the very start of every load, turning legacy names into current ones before anything else sees the config. With no file present, nothing is translated and loading behaves as it always did.

The order it works in

Each step depends on the ones before it - moves target already-renamed sections, companions see converted key names:

1. `sections` - rename top-level blocks
2. `properties` - explicit per-property renames
3. `auto_case_normalization` - automatic case and underscore fixes
4. `value_map` - normalize stored values
5. `moves` - relocate a property into another section
6. `remove_unused_devices` - drop legacy blocks not in use
7. `escape_carriage_returns` - turn control characters into the file's `\r` marker
8. `device_labels` / `device_templates` - friendly names, buttons, labels, modules
9. `companion_keys` - create partner keys
10. `defaults` - inject anything still missing

The rule groups

sections - a list of `{ match, rename_to }`, where `match` is a whole-name regex and `$1`..`$9` insert capture groups. First match wins.

```
"sections": [
  { "match": "CAMERA(\\d+)DEVICE", "rename_to": "CAMERADEVICE_$1" },
  { "match": "CAMERADEVICE",       "rename_to": "CAMERADEVICE_1" },
  { "match": "SYSTEM|SYSTEMSETUP", "rename_to": "SYSTEM_SETUP" }
]
```

properties - a flat old-name to new-name map applied inside every section. Always wins over automatic normalization.

auto_case_normalization - when true, a legacy property whose lowercased, underscore-stripped form matches a known current key is renamed automatically, so `Output_Proj1`, `GVE_BLDG` and `COMTYPE` need no entries of their own. Only spellings that differ structurally need one.

value_map - keyed by the *new* property name. Mapped values keep their JSON type, so a string can become a boolean.

```
"value_map": {
  "active_notifications": { "True": true, "False": false },
  "com_type": { "NETWORK": "Network", "SERIAL": "Serial" }
}
```

moves - relocate a property into another section, capture groups usable in the target name. Legacy files kept per-device GVE IDs in `SYSTEM`; this is how they reach each device.

remove_unused_devices + **device_counts** - legacy blocks whose family count says they aren't in use are removed rather than converted. Only blocks that were *renamed* in step 1 are eligible, so a current-style template that deliberately carries every block keeps them all.

escape_carriage_returns - real control characters inside strings become the two-character marker the processor reads, so an outlet name arrives as `"Doc\\rCam"` and breaks over two lines on the panel button.

companion_keys - guarantee a partner key exists for everything matching a pattern, with the number carried over. Every `power1_outlet_N` gets a `power1_outlet_N_action`, only for outlets that exist, and an existing value is never overwritten.

device_labels / **device_templates** - friendly fill-in when converting: `name` built as "Label - model", numbered only when the family has more than one unit; `btn_name` and `lbl_name` carried over by device number; `module` and `keep_alive_trigger` carried over when the legacy model matches a template block.

defaults - after everything else, any listed key still missing is injected. `{n}` in a value becomes the device number.

When something looks wrong

What you see	What it usually is
A field shows a raw key name and a plain text box	No schema entry for it. **Schema → Coverage → Describe**.
A red outline on two fields	A consistency rule says they disagree. The red helper line says which.
A device tab you didn't expect, or one missing	The family's `dev_` count on the Wizard, or the family list under **Schema → Device families**.
The AV Flow is missing a lead	Press **Draw the routing from config** and read the reason. Usually a blank config number or a connector the catalog doesn't list.
A run drawn straight from a DTP output into an HDMI socket	The extender rule for that pair was removed. **Flow Rules → Extenders**.
Nothing plugged into the USB switcher	The USB rule names boxes this room doesn't have, or their catalog entries have no USB connector.
An outlet's mains lead isn't drawn	Two boxes answered to the name equally well, so nothing was drawn. Add an entry under **Flow Rules → Outlet names**.
A box on the drawing that costs nothing	Its model isn't in the catalog. The Cost tab counts these for you.
A drawing that no longer matches the room	**Recreate from config**, on either the AV Flow or the Schematic tab.
The wrong room in the SFTP dialog	The active deployment target clears when you open a different config - pick the room again.
A vendor rule that claims nothing	It's matched exactly. Use **Pick from the job** rather than typing - "Extron Electronics" is not "Extron".
A part on no vendor's list	Nothing claimed it. The **Untagged** chip on Equipment is the to-do list; add a rule, or pin the vendor on the line.
A quote request that isn't on the timeline	Only vendors with a date recorded are - press **RFQ sent** on the vendor card.
A PO number missing from a dropdown	It's swept from the rows, the vendors, the parts and the deliveries. A number nobody has typed anywhere yet won't be there.
A refresh figure that looks like an old price	The model is retired and names no successor. **Catalog → Retired → Replaced by…**.
A whole category budgeted at nothing	No base cost card for it. **Campus → Current models** prices it off a model you pick.

What you see	What it usually is
A base cost that's years out of date	The card says when it was set. Re-price it on **Campus → Current models**.
The date rail is a row of overlapping dots	It's fitted to the whole job. Zoom in - the readout says how much is in the frame.
A responsibility line reading NOBODY	Nobody has been named for it. That's the blank the sheet exists to catch, not a bug.

Which file does what

File	Read when	Written by	If it's missing
`config.json` (template)	New Config	Never	New Config can't run
`ui_schema.json`	Startup, Reload Schema	Schema tab	Built-in field definitions
`key_map.json`	Every load	Text editor	Nothing is translated
`av_devices.json`	Startup, Reload Catalog	Catalog tab	Built-in models, no prices, no lives, no successors
`av_flow_rules.json`	Startup, Reload Rules	Flow Rules tab	Built-in drawing rules
`processors.json`	Startup	Text editor	No room list in the SFTP dialogs
`buildings.json`	Startup	Text editor	No building names or codes
`labor_rates.json`	Startup	Cost tab	Built-in roles, no rates
`base_costs.json`	Startup	Cost tab, campus **Current models**	Every category unpriced
`app_config.json`	Startup	App Config tab	First-run setup runs

Keeping the guides honest

There are two documents, and both are built from Markdown in the repository:

Document	Source
This one - *Room Config Builder, operation guide*	`documentation/Room_Config_Builder_Guide.md`
The Beginner's Guide to Room Config - the first week, in the order you hit it	`documentation/Beginners_Guide_to_Room_Config.md`

Edit the source and run:

```
python tools/build_guide.py
```

which rewrites the `.pdf` and the `.docx` of **both**, in place, so the file names people have bookmarked keep working. Name one to build only it:

```
python tools/build_guide.py beginners
```

Keeping the sources in the repo means a change to the app and the change to its documentation can travel in the same commit, and anybody can see what moved. It also means neither can quietly go stale: a `.docx` somebody edits by hand and exports has nothing in the repo saying it is behind, and nobody can diff it.

There is a third place all of this lives, and it is the one most people will actually read: the **help book inside the app**, on F1. It is written in `lib/help_content.dart` and travels with the build, so it cannot be a version behind on somebody's desktop. When a feature changes, all three want a look.